Rupa

Plan, guide, and tutorial for a renderer layer for Claude Code

Vajra Sarpa · October 15, 2026

CONTENTS

- About this document

Part 1: Plan
- The problem
- The idea
- Goals and non-goals
- Architecture
- Key decisions
- Milestones
- Success metrics
- Risks

Part 2: Guide
- The contract
- Input format
- Themes
- Verification
- Plugin anatomy
- Token discipline
- Measuring the savings
- Well-Architected lens

Part 3: Tutorial
- Before you start
- Step 1: Scaffold the project
- Step 2: The document model
- Step 3: The renderer and themes
- Step 4: The CLI contract
- Step 5: Install it as a command
- Step 6: Continuous integration
- Step 7: The Claude Code plugin
- Step 8: Verification
- Step 9: Measure
- Step 10: What to build next

Turning the build into a showcase

Appendix: Verification record
- Pass 1: the first delivered version
- Pass 2: after adding verification
- Final pass

About this document

Rupa (Sanskrit *rūpa*, "form") is a small command-line renderer. Claude writes content as Markdown; Rupa owns the form, turning that Markdown into a styled HTML page or PDF. A thin Claude Code plugin teaches Claude to use it.

The document has three parts plus a short note on presenting the build:

- **Part 1, Plan:** what to build, why, the decisions, milestones and risks.
- **Part 2, Guide:** the reference for how Rupa works once it exists.
- **Part 3, Tutorial:** build it step by step, from an empty folder to a tested CLI, a validated plugin, a verification loop and a benchmark.

Everything in Part 3 was built and run before this was written: 40 tests pass, `claude plugin validate` passes, and Claude Code loads the skill and uses it to render and verify a PDF on its own. This document was rendered by that prototype from one Markdown file, then checked with `rupa verify` and a visual review of its sample pages. The appendix records what that review found.

Part 1: Plan

The problem

When Claude Code produces an HTML page or PDF, it writes the markup itself: every tag, every CSS rule, or a script that builds the PDF. Four costs follow.

- **Turns.** Finding a PDF tool, installing it, rendering, and debugging the result each take a turn, and every turn re-reads the whole conversation from cache. In the one run measured during the build, cache reads were about 99% of the tokens processed, so turn count is what drives cost.
- **Output tokens.** Markup and styling are a large share of what Claude writes. They are the slowest tokens, but a small share of the total.
- **Revision cost.** A one-line content fix often regenerates the whole file, styling included.
- **Inconsistency.** Each document is styled from scratch, so two reports never look quite the same, and render bugs are fixed in one file at a time.

The idea

Split content from form. Claude emits compact Markdown with a few frontmatter keys. Rupa, a deterministic and tested program, applies the theme and produces the file. Rupa prints one status line, so Claude doesn't read the rendered output back by accident.

Then check the result where it matters, because people read these documents and people like pretty things. Rupa runs code checks on the PDF and prepares a small set of page images, each next to the matching page of the theme's reference. Claude looks at those deliberately. Tokens are saved on generation and spent on review.

Without Rupa	With Rupa
Claude writes HTML, CSS, PDF code	Claude writes Markdown
Styling re-emitted on every revision	Styling lives in Rupa themes
Claude re-reads the whole output to check it, or doesn't check	Code checks every page; Claude looks at a few sampled pages next to a reference
Every document styled ad hoc	One theme, applied the same way every time

Goals and non-goals

Goals

1. Markdown plus frontmatter in, HTML or PDF out, from stdin or a file.
2. stdout carries one status line (or one JSON object), never the document.
3. Clear exit codes so Claude can fix input without guessing.
4. PDF body text is solid black (#000000, full opacity).
5. A Claude Code plugin under about 30 lines of instructions, with no always-on MCP tool definitions.
6. Every PDF verified twice: code checks for what can be measured, and a visual review against the theme's reference pages for what can't.
7. A repeatable benchmark that measures turns and cost per finished document instead of assuming them.

Non-goals for v1

- A layout language. If Markdown cannot express it, it waits for a later version.
- DOCX or PPTX output. They can be added as new targets behind the same contract.
- Remote assets. PDF rendering refuses network fetches by design.

Architecture

The pipeline is linear and each stage has one job.

```
Markdown + frontmatter   (stdin or a file)
        │
        ▼
parse()                  document.py   → Document(meta, body_html, headings)
        │
        ▼
to_html()                render.py + themes/*.css
        │
        ├──► .html file
        └──► to_pdf()    WeasyPrint    → .pdf file
                                              │
                                              ▼
verify()                 verify.py     → checks + sheets + verify.json
        │
        ▼
Claude reads the sheets, judges them against the rubric, fixes the Markdown
```

```
stdout: one status line          exit code: 0 / 2 / 3 / 4 / 5
```

Module	Responsibility	Knows about styling?
`document.py`	Parse frontmatter, Markdown, headings	No
`render.py`	HTML template, theme CSS, PDF	Yes, and only here
`themes/*.css`	All visual decisions	Yes
`verify.py`	PDF checks, page types, review sheets	Reads the result, never styles it
`references/`	Reference pages for each theme and page type	They are the theme's picture of itself
`cli.py`	Arguments, I/O, status line, exit codes	No
`plugin/`	Teach Claude the contract	No

Key decisions

Decision	Choice	Why	Rejected
Interface	CLI over stdin/stdout	Works in Claude Code, CI, scripts and any harness; zero context cost until called	MCP server: tool schemas sit in context every turn
Input format	Markdown + YAML frontmatter	Claude writes it fluently; few tokens; no schema to teach	Custom JSON schema: more prompt overhead and more retries
PDF engine	WeasyPrint	Pure Python install, real CSS paged media (page numbers, running titles)	Headless Chrome: heavy dependency; LaTeX: a second styling system
Markdown parser	markdown-it-py	CommonMark compliant, plugin system, token stream for headings	Python-Markdown: weaker spec compliance
CLI library	argparse	No dependency, enough for six flags	Typer/Click: nice, not needed yet
Raw HTML in input	Escaped	Keeps Claude from smuggling styling back in; safer output	Pass-through
Plugin form	One skill + one hook	Skill body loads only when triggered	Many slash commands

Decision	Choice	Why	Rejected
Verification	Code checks, then vision on sampled pages beside reference pages	Code is exact and free; vision judges what code can't. Sampling by page type keeps the cost tied to variety, not length	Vision on every page (~85k tokens for 31 pages); pixel diffs (content always differs from the reference)
Who looks at the images	Claude, through the skill	Rupa stays offline with no API key; Claude is already a vision model	Rupa calling a model API itself

Milestones

Estimates assume evenings and weekends.

#	Milestone	Done when	Estimate
M0	Scaffold	`uv sync` and an empty test run pass	1 hour
M1	Parser	Document model and 8 parser tests pass	2–3 hours
M2	Renderer	HTML and PDF from a sample; PDF text forced black	3–4 hours
M3	CLI contract	Status line, `--json`, exit codes, CLI tests pass	2 hours
M4	Plugin	`claude plugin validate` passes; skill loads with `--plugin-dir`	1–2 hours
M5	Verify	`rupa verify` checks and sheets; reference pages built; skill runs the review loop	4–6 hours
M6	Measure	Pilot of one run per arm, then three runs per arm; medians recorded	2 hours plus about $2–6 of API use
M7	Ship	CI green on GitHub Actions; `uv tool install` from the repo works	1–2 hours

Success metrics

- **Primary:** median turns and median cost (`total_cost_usd`) per finished document, Rupa against baseline. Turns come first because each one re-reads the whole context, so fewer turns is where most of the saving should come from. Report the numbers you get, whichever way they go.

- **Secondary:** median cache reads and output tokens, and the success rate (runs that produced the PDF).
- **Verification overhead:** the difference between the `rupa` and `rupa-verify` arms, which is what the visual review costs per document.
- **Quality:** all tests green; the same input renders the same output every time; every delivered PDF passes `rupa verify` and a visual review with no open failures.

Risks

Risk	Likelihood	Impact	Mitigation
Savings smaller than expected	Medium	Medium	Pilot in M6 before investing in themes; judge on turns and cost, not output tokens alone
Vision verdicts vary between runs	Medium	Medium	Fixed rubric, one structured verdict per sheet, code checks for anything measurable, two fix rounds at most
Code checks misread a page (page type, fill)	Medium	Low	Unit tests on synthetic pages; flagged pages always go to the visual review, so a wrong flag costs one sheet
Claude ignores the skill and hand-writes HTML	Medium	High	Short, direct skill description; check with the benchmark transcripts
WeasyPrint system libraries missing on a machine	Medium	Low	Exit code 4 with a clear message; install notes in the tutorial; CI installs them
Markdown can't express a layout the user wants	High over time	Medium	Add themes and small, explicit extensions; never accept raw HTML
Library API changes (this happened during the build: WeasyPrint 70 replaced its URL fetcher API)	Medium	Low	Pin minimum versions; tests cover the behavior, not the library call

Part 2: Guide

The contract

Rupa has one contract, and the plugin, tests and benchmark all depend on it.

Channel	Carries
stdin or file argument	Markdown with optional YAML frontmatter
`-o PATH`	The rendered file. Format comes from the extension, or `--to html` / `--to pdf`
stdout	One status line, or one JSON object with `--json`. Never the document
stderr	One error line on failure
exit code	`0` ok, `2` usage, `3` bad input, `4` render failure, `5` verify checks failed

Two subcommands follow the same rules:

Command	Does	Prints
`rupa verify OUT.pdf [--wants F]`	Runs the checks, writes sheets and `verify.json`	One line: checks passed, failures, sheet count
`rupa refs [--theme T]`	Rebuilds a theme's reference pages from the bundled sample	One line: which sample page became each reference

Example:

```
$ rupa notes.md -o out/notes.pdf
rupa: wrote out/notes.pdf (pdf, 3 pages, 48,211 bytes)

$ rupa notes.md -o out/notes.pdf --json
{"ok": true, "path": "out/notes.pdf", "format": "pdf", "pages": 3, "bytes": 48211}
```

Input format

Frontmatter is optional. Only these keys are accepted; anything else is exit code 3, so typos fail loudly instead of being ignored.

Key	Type	Effect
`title`	string	Cover title and PDF running footer. Defaults to the first heading
`subtitle`	string	Line under the title
`author`	string	Byline
`date`	string	Byline, after the author
`theme`	string	Theme name. Defaults to `default`; `--theme` overrides it
`toc`	bool	Table of contents from `#` and `##` headings. Defaults to true

Body conventions:

- `#` is a part. In PDF each part starts a new page.
- `##` is a section, `###` a subsection. Sections get anchors automatically.
- Fenced code blocks take a language tag for highlighting.
- Tables use GitHub pipe syntax.
- Raw HTML is escaped and shown as text.
- Images must be local files. Remote URLs fail the PDF render with exit code 4.

Themes

A theme is one CSS file in `src/rupa/themes/`. The PDF target appends `print.css` on top of it, which sets the page size, margins, footer, page breaks, and forces all text to solid black:

```
body, p, li, td, th, h1, h2, h3, h4, blockquote, code, pre,
.subtitle, .byline, .toc a, a {
  color: #000000 !important;
  opacity: 1 !important;
}
.hl * { color: #000000 !important; font-style: normal !important; }
```

The cyan accent survives only on rules and borders, never on text. Syntax highlighting is turned off in PDF for the same reason.

Code blocks use `white-space: pre-wrap` and nothing else. WeasyPrint treats `word-wrap: break-word` as permission to break anywhere, which split `solid` into `s` and `olid` in an earlier render of this document. The visual review caught it; the code checks couldn't.

To add a theme, copy `default.css` to `newname.css`, change the variables at the top, add the name to `THEMES` in `render.py`, and run `rupa refs --theme newname` to build its reference pages.

Verification

Rendering proves the file exists. Verification checks that it's right, in two tiers.

Tier 1: code checks. `rupa verify` reads the PDF with pdfplumber and checks what can be measured exactly. Each check is free, repeatable and reports page numbers.

Check	Fails when
`text-color`	Any character isn't the wanted color (#000000 by default)
`page-size`	A page isn't Letter (or A4, if the wants file says so)
`max-pages`	The document is longer than the limit
`blank-page`	A page has no text
`near-empty`	A page is under 20% full (cover and last page exempt)
`sparse`	A page is under 40% full and the next page doesn't start a new part
`heading-at-bottom`	A heading is the last line on a page
`overflow`	Text runs past the side margins

Tier 2: visual review. Code can't tell whether a page looks right. For that, `rupa verify` classifies each page as cover, prose, table or code. It then writes a sheet image for every flagged page plus one page of each type, with the theme's reference page on the left and the rendered page on the right. Claude opens those sheets and judges each one against a fixed rubric: hierarchy, spacing, overflow, page balance and style.

The wants file

A `wants.yaml` holds what a finished PDF must satisfy. Every key is optional, unknown keys are an error, and `wants.example.yaml` in the repo spells out the defaults. A project that needs A4 and a 10-page limit writes two lines:

```
page_size: a4
max_pages: 10
```

Reference pages

A theme's reference pages are its picture of itself. `rupa refs` renders a bundled sample document with the theme and saves the most typical page of each type as a PNG inside the package. Rebuild them whenever the theme's CSS changes, and commit them with it. A sheet without a reference still shows the page, labeled so.

What it costs

An image costs roughly its width times height divided by 750 in input tokens. A sheet at the default 90 DPI is about 1,530 × 1,020 pixels, or about 2,000 tokens.

How Claude reviews the 31-page version of this document	Images	Input tokens, approx.
Every page at full resolution	31	85,000
`rupa verify` : flagged pages plus one per type	5	10,000
Code checks only	0	under 100

The sampled review scales with how varied the document is, not how long it is. That is the right place to spend tokens: the reader sees the result.

The loop

The skill tells Claude to render, verify, open the sheets, and write one verdict per sheet. Claude fixes failures in the Markdown only: split a wide table, shorten a code line, move content around an awkward break. Then it renders and verifies again, at most twice. A problem that can only be fixed in CSS is reported as a theme issue rather than patched, so one document's quirks don't leak into the theme.

Plugin anatomy

The plugin is deliberately small. Claude Code only needs to know that Rupa exists and how to call it.

```
plugin/
├── .claude-plugin/
│   └── plugin.json        # name, version, description, author
├── skills/
│   └── render/
│       └── SKILL.md       # the whole instruction set, ~40 lines
```

```
└── hooks/
    └── hooks.json        # SessionStart: warn if rupa is not on PATH
```

What each part costs in context:

Component	When it loads	Cost
Skill name and description	Every session	One line
Skill body	Only when the skill is triggered	About 40 lines
Review sheets	Only when Claude opens them during verification	About 2,000 tokens each
SessionStart hook	Session start	Prints nothing if `rupa` is installed
MCP server	Not used	Would be every turn

Claude Code namespaces plugin components under the plugin name, so the skill appears as `rupa:render`.

Two alternatives you may want later:

- `bin/` **directory.** Files in a plugin's `bin/` are put on the Bash tool's PATH while the plugin is enabled, so you could ship a `rupa` wrapper inside the plugin. The catch: claude.ai and Cowork don't install a plugin that has a top-level `bin/` directory, so this limits where the plugin can be used. Keep the CLI installed separately with `uv tool install` unless you only care about Claude Code.
- **MCP server.** Worth it only if Rupa grows state or a live preview. Until then it adds per-turn context for nothing.

Token discipline

The savings come from what Claude does not do. The skill enforces four rules:

1. Write Markdown, not markup.
2. Never put styling in the content. A different look is a theme change.
3. Read the status line, not the output file. Look at pages only through the review sheets, where looking is the point.
4. On failure, read the one stderr line and fix the input.

The fourth rule depends on Rupa's errors being specific. "unknown frontmatter keys: ['colour']" lets Claude fix the problem in one edit. A stack trace would cost a turn and a lot of tokens.

Measuring the savings

Where the cost goes

One run from the build, not a benchmark: asked for a one-page PDF summary of the six Well-Architected pillars, Claude Code with the plugin rendered, ran `rupa verify`, opened the sheet, and fixed the Markdown once to get the document to one page. No HTML or CSS was written by hand.

Measure	Value
Turns	16
Cache reads	636,178 tokens
Cache writes	57,643 tokens
Uncached input	28 tokens
Output	3,651 tokens
Total cost	$0.33

Output was about half a percent of the tokens processed. Every turn re-reads the conversation, so cost tracks turns. That changes what Rupa should be judged on: not mainly smaller output, but fewer turns. No hunting for a PDF tool, no render debugging, and one-line errors that are fixed in one edit.

Three arms

`bench/bench.sh` runs the same task in three arms through Claude Code's print mode (`claude -p ... --output-format json`):

Arm	Setup	Answers
`baseline`	No plugin	What it costs today
`rupa`	Plugin with the skill's Verify section removed	What rendering through Rupa saves
`rupa-verify`	Plugin as shipped	What the visual review adds back

The render-only arm uses a copy of the plugin made at run time, so all three arms get exactly the same prompt. Each run starts in a fresh empty directory, so no arm can read the Rupa repo. `bench/summarize.py` prints one row per run and the medians per arm, with turns and cache reads first, then a line comparing each Rupa arm to the baseline.

What it will cost

Estimates on the default model, from the run above:

Arm	Per run, approx.
`baseline`	$0.40–1.00; wide, because it depends on whether Claude detours to find and install a PDF tool
`rupa`	$0.25–0.40
`rupa-verify`	$0.30–0.50

A pilot of one run per arm is about $1–2. Three runs per arm is about $3–6. A pricier model scales these proportionally.

How to read the results

- Compare medians, not single runs. Agent runs vary.
- Judge on turns and cost first. Output tokens are secondary.
- Count failed runs (no PDF) separately; a cheap run that produced nothing isn't a saving.
- Open one transcript per arm. Confirm the Rupa arms used Rupa and the render-only arm didn't verify. If an arm misbehaved, fix that before trusting any number.
- Run all arms on the same model on the same day, and pin the model with `MODEL=...`.

Well-Architected lens

The AWS Well-Architected pillars apply even to a local tool.

Pillar	How Rupa addresses it
Operational excellence	One status line and stable exit codes make it scriptable; CI runs tests and a smoke render on every push
Security	Raw HTML escaped; PDF render limited to `file:` and `data:` URLs; Jinja autoescaping; no secrets anywhere

Pillar	How Rupa addresses it
Reliability	Input validated before render; every failure maps to an exit code and one message; every PDF checked by `rupa verify`
Performance efficiency	No network, no browser process; this 30-plus-page document renders to PDF in a few seconds
Cost optimization and sustainability	Fewer turns and output tokens per document, measured rather than claimed; vision spent on sampled pages, not every page

Part 3: Tutorial

Before you start

You need:

- Python 3.11 or newer and uv.
- WeasyPrint's system libraries (Pango). On Ubuntu or Debian: `sudo apt install libpango-1.0-0 libpangoft2-1.0-0`. On macOS: `brew install pango`. On Windows, use WSL.
- Claude Code, for steps 7 to 9.

The finished project looks like this:

```
rupa/
├── pyproject.toml
├── README.md
├── src/rupa/
│   ├── __init__.py
│   ├── document.py
│   ├── render.py
│   ├── cli.py
│   ├── verify.py
│   ├── themes/
│   │   ├── default.css
│   │   └── print.css
│   └── references/
│       ├── sample.md     # source for every theme's reference pages
│       └── default/      # cover.png, prose.png, table.png, code.png
├── tests/
│   ├── test_document.py
│   ├── test_cli.py
│   └── test_verify.py
├── plugin/               # Claude Code plugin
├── wants.example.yaml
├── bench/
│   ├── bench.sh
│   ├── summarize.py
│   └── fake/claude     # stand-in for testing the plumbing
└── .github/workflows/ci.yml
```

Step 1: Scaffold the project

```
mkdir rupa && cd rupa
git init
mkdir -p src/rupa/themes tests plugin bench .github/workflows
```

Create `pyproject.toml`. The `[project.scripts]` entry is what makes `rupa` a command once installed. The `pythonpath` setting lets pytest import the package without installing it.

```
[project]
name = "rupa"
version = "0.1.0"
description = "Render Markdown + frontmatter from stdin into styled HTML or PDF."
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "markdown-it-py>=3.0",
    "mdit-py-plugins>=0.4",
    "jinja2>=3.1",
    "pyyaml>=6.0",
    "pygments>=2.17",
    "weasyprint>=70",
    "pdfplumber>=0.11",
]

[project.scripts]
rupa = "rupa.cli:main"

[dependency-groups]
dev = ["pytest>=8"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/rupa"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
```

Create `src/rupa/__init__.py`:

```
"""Rupa: a renderer layer. Content in on stdin, styled HTML or PDF out."""

__version__ = "0.1.0"
```

Then install everything into a local environment:

```
uv sync
```

Checkpoint: `uv run pytest` runs and reports that no tests were collected.

Step 2: The document model

This module turns text into a `Document`: metadata, body HTML and a list of headings. It knows nothing about styling. Note three choices that matter later:

- `html: False` escapes raw HTML in the input.
- Unknown frontmatter keys are an error, not silently ignored.
- Headings are collected from the token stream, which is what the table of contents uses.

`src/rupa/document.py`:

```
"""Parse Markdown + YAML frontmatter into a Document model.

The model is the contract between Claude and Rupa: Claude writes plain
Markdown, Rupa owns every styling decision downstream of this file.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field

import yaml
from markdown_it import MarkdownIt
from mdit_py_plugins.anchors import anchors_plugin
from mdit_py_plugins.front_matter import front_matter_plugin
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import TextLexer, get_lexer_by_name
```

```
from pygments.util import ClassNotFound

ALLOWED_META = {"title", "subtitle", "author", "date", "theme", "toc"}


class InputError(ValueError):
    """Raised when the input cannot be turned into a Document."""


@dataclass
class Heading:
    level: int
    text: str
    anchor: str


@dataclass
class Document:
    meta: dict
    body_html: str
    headings: list[Heading] = field(default_factory=list)

    @property
    def title(self) -> str:
        if self.meta.get("title"):
            return str(self.meta["title"])
        return self.headings[0].text if self.headings else "Untitled"


def _highlight(code: str, lang: str, _attrs: str) -> str:
    try:
        lexer = get_lexer_by_name(lang) if lang else TextLexer()
    except ClassNotFound:
        lexer = TextLexer()
    return highlight(code, lexer, HtmlFormatter(nowrap=True))


def _build_parser() -> MarkdownIt:
    # html=False: raw HTML in the input is escaped, never passed through.
    md = MarkdownIt("commonmark", {"html": False, "highlight": _highlight}).enable(
        "table"
    )
    md.use(front_matter_plugin)
    md.use(anchors_plugin, min_level=1, max_level=3)
    return md


def parse(text: str) -> Document:
    if not text.strip():
```

```
        raise InputError("empty input: nothing to render")

    md = _build_parser()
    tokens = md.parse(text)

    meta: dict = {}
    headings: list[Heading] = []
    for i, tok in enumerate(tokens):
        if tok.type == "front_matter":
            try:
                loaded = yaml.safe_load(tok.content) or {}
            except yaml.YAMLError as exc:
                raise InputError(f"bad frontmatter: {exc}") from exc
            if not isinstance(loaded, dict):
                raise InputError("frontmatter must be a YAML mapping")
            unknown = set(loaded) - ALLOWED_META
            if unknown:
                raise InputError(f"unknown frontmatter keys: {sorted(unknown)}")
            meta = loaded
        elif tok.type == "heading_open":
            inline = tokens[i + 1]
            headings.append(
                Heading(
                    level=int(tok.tag[1]),
                    text=inline.content,
                    anchor=tok.attrs.get("id", ""),
                )
            )

    body = md.renderer.render(tokens, md.options, {})
    # Fenced code gets Pygments spans; mark the block for theme CSS.
    body = re.sub(r"<pre><code", '<pre class="hl"><code', body)
    return Document(meta=meta, body_html=body, headings=headings)
```

Write the tests before moving on. `tests/test_document.py`:

```
import pytest

from rupa.document import InputError, parse

SAMPLE = """---
title: Sample
author: Vajra Sarpa
---

## First section
```

```
Hello **world**.

```python
print("hi")
```
"""

def test_frontmatter_becomes_meta():
    doc = parse(SAMPLE)
    assert doc.meta == {"title": "Sample", "author": "Vajra Sarpa"}
    assert doc.title == "Sample"

def test_headings_are_collected_with_anchors():
    doc = parse(SAMPLE)
    assert [(h.level, h.text) for h in doc.headings] == [(2, "First section")]
    assert doc.headings[0].anchor == "first-section"

def test_code_is_highlighted():
    doc = parse(SAMPLE)
    assert '<pre class="hl">' in doc.body_html
    assert "<span" in doc.body_html

def test_title_falls_back_to_first_heading():
    assert parse("# From heading\n\ntext").title == "From heading"

def test_empty_input_is_rejected():
    with pytest.raises(InputError, match="empty"):
        parse("   \n")

def test_unknown_frontmatter_key_is_rejected():
    with pytest.raises(InputError, match="unknown frontmatter"):
        parse("---\ncolour: red\n---\n\ntext")

def test_raw_html_is_escaped():
    doc = parse("<script>alert(1)</script>\n")
    assert "<script>" not in doc.body_html
    assert "<script>" in doc.body_html

def test_non_mapping_frontmatter_is_rejected():
    with pytest.raises(InputError, match="mapping"):
        parse("---\n- a\n- b\n---\n\ntext")
```

Checkpoint: `uv run pytest tests/test_document.py -q` shows 8 passed.

Step 3: The renderer and themes

All styling decisions live in `render.py` and the CSS files. The HTML template is inline so the package has one less file to ship. The PDF function restricts resource fetching to local files and `data:` URLs.

One trap worth knowing: WeasyPrint's `FatalURLFetchingError` inherits from `BaseException`, not `Exception`, so a plain `except Exception` does not catch it. That is why the code names it explicitly. This was found by a failing test during the build.

`src/rupa/render.py`:

```
"""Turn a Document into HTML or PDF bytes. All styling lives here and in themes/."""

from __future__ import annotations

from importlib import resources

from jinja2 import Environment, select_autoescape
from pygments.formatters import HtmlFormatter

from .document import Document

TEMPLATE = """<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>{{ title }}</title>
<style>{{ css | safe }}</style>
</head>
<body class="{{ target }}">
<header class="cover">
  <h1 class="doc-title">{{ title }}</h1>
  {% if meta.subtitle %}<p class="subtitle">{{ meta.subtitle }}</p>{% endif %}
  {% if meta.author or meta.date %}
  <p class="byline">{{ meta.author or "" }}{% if meta.author and meta.date %} &middot; {%
endif %}{{ meta.date or "" }}</p>
  {% endif %}
</header>
{% if toc %}
<nav class="toc"><h2>Contents</h2><ol>
{% for h in toc %}<li class="l{{ h.level }}"><a href="#{{ h.anchor }}">{{ h.text }}</a></li>
```

```
{% endfor %}</ol></nav>
{% endif %}
<main>
{{ body | safe }}
</main>
</body>
</html>
"""

THEMES = ("default",)


class RenderError(RuntimeError):
    """Raised when a target cannot be produced."""


def _theme_css(theme: str, target: str) -> str:
    if theme not in THEMES:
        raise RenderError(f"unknown theme '{theme}' (have: {', '.join(THEMES)})")
    pkg = resources.files("rupa") / "themes"
    css = (pkg / f"{theme}.css").read_text(encoding="utf-8")
    if target == "pdf":
        css += "\n" + (pkg / "print.css").read_text(encoding="utf-8")
    css += "\n" + HtmlFormatter(style="default").get_style_defs(".hl")
    return css


def to_html(doc: Document, theme: str = "default", target: str = "html") -> str:
    env = Environment(autoescape=select_autoescape(default=True))
    toc_on = doc.meta.get("toc", True)
    # H1s act as parts, H2s as sections. An H1 that repeats the title is skipped.
    toc = (
        [h for h in doc.headings if h.level <= 2 and h.text != doc.title]
        if toc_on
        else []
    )
    return env.from_string(TEMPLATE).render(
        title=doc.title,
        meta=doc.meta,
        toc=toc,
        body=doc.body_html,
        css=_theme_css(theme, target),
        target=target,
    )


def to_pdf(doc: Document, theme: str = "default") -> tuple[bytes, int]:
    """Return (pdf_bytes, page_count)."""
    try:
```

```
        from weasyprint import HTML, URLFetcher
        from weasyprint.urls import FatalURLFetchingError
    except ImportError as exc:  # pragma: no cover - depends on system libs
        raise RenderError("PDF needs WeasyPrint and its system libraries") from exc
    # Local files and data: URIs only. A remote URL fails the render instead of
    # silently fetching from the network.
    fetcher = URLFetcher(allowed_protocols={"file", "data"}, fail_on_errors=True)
    try:
        rendered = HTML(
            string=to_html(doc, theme, target="pdf"), url_fetcher=fetcher
        ).render()
    except FatalURLFetchingError as exc:  # subclasses BaseException, not Exception
        raise RenderError(f"blocked or missing resource: {exc}") from exc
    return rendered.write_pdf(), len(rendered.pages)
```

The screen theme, `src/rupa/themes/default.css`:

```
/* Rupa default theme: slate ink, one cyan accent. Screen first. */
:root {
  --ink: #0f172a;
  --muted: #334155;
  --rule: #cbd5e1;
  --accent: #0e7490;
  --code-bg: #f1f5f9;
  --bg: #ffffff;
  --measure: 46rem;
}
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --ink: #e2e8f0; --muted: #94a3b8; --rule: #334155;
    --accent: #22d3ee; --code-bg: #1e293b; --bg: #0f172a;
  }
}
* { box-sizing: border-box; }
html { background: var(--bg); }
body {
  margin: 0 auto;
  padding: 3rem 1rem 5rem;
  max-width: var(--measure);
  font: 400 16px/1.65 Inter, "Helvetica Neue", Arial, sans-serif;
  color: var(--ink);
  background: var(--bg);
}

/* Cover and table of contents */
.cover {
  border-bottom: 3px solid var(--accent);
```

```
  padding-bottom: 1.25rem;
  margin-bottom: 2rem;
}
.doc-title {
  font-size: 2.3rem;
  line-height: 1.15;
  margin: 0 0 .4rem;
  letter-spacing: -0.02em;
}
.subtitle { font-size: 1.15rem; color: var(--muted); margin: 0 0 .6rem; }
.byline { font-size: .9rem; color: var(--muted); margin: 0; }
.toc {
  margin: 0 0 2.5rem;
  padding: 1rem 1.25rem;
  border: 1px solid var(--rule);
  border-radius: 8px;
}
.toc h2 {
  font-size: .8rem;
  text-transform: uppercase;
  letter-spacing: .08em;
  margin: 0 0 .5rem;
  border: 0;
  padding: 0;
}
.toc ol { margin: 0; padding-left: 1.25rem; }
.toc a { color: var(--ink); text-decoration: none; }
.toc li.l1 {
  font-weight: 650;
  margin-top: .5rem;
  list-style: none;
  margin-left: -1.25rem;
}
.toc li.l2 { list-style: none; }
.toc a:hover { color: var(--accent); }
main > h1:first-child { display: none; } /* the cover already shows the title */

/* Headings and body */
h1, h2, h3 { line-height: 1.25; letter-spacing: -0.01em; }
h1 { font-size: 1.9rem; margin: 3rem 0 1rem; }
h2 {
  font-size: 1.45rem;
  margin: 2.75rem 0 .9rem;
  padding-bottom: .35rem;
  border-bottom: 1px solid var(--rule);
}
h3 { font-size: 1.12rem; margin: 2rem 0 .6rem; }
p, ul, ol { margin: 0 0 1rem; }
li { margin: .2rem 0; }
```

```
a { color: var(--accent); }
strong { font-weight: 650; }
blockquote {
  margin: 1.25rem 0;
  padding: .6rem 1rem;
  border-left: 4px solid var(--accent);
  background: var(--code-bg);
}
blockquote p:last-child { margin-bottom: 0; }

/* Code */
code {
  font: 0.88em/1.5 "JetBrains Mono", "DejaVu Sans Mono", Menlo, monospace;
  background: var(--code-bg);
  padding: .1em .35em;
  border-radius: 4px;
}
pre {
  background: var(--code-bg);
  padding: .9rem 1rem;
  border-radius: 8px;
  overflow-x: auto;
  border: 1px solid var(--rule);
}
pre code { background: none; padding: 0; font-size: .82rem; }

/* Tables */
table {
  border-collapse: collapse;
  width: 100%;
  margin: 0 0 1.25rem;
  font-size: .92rem;
  display: block;
  overflow-x: auto;
}
th, td {
  border: 1px solid var(--rule);
  padding: .45rem .6rem;
  text-align: left;
  vertical-align: top;
}
th { background: var(--code-bg); font-weight: 650; }
hr { border: 0; border-top: 1px solid var(--rule); margin: 2.5rem 0; }
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) .hl * { color: inherit !important; }
}
```

The PDF overrides, `src/rupa/themes/print.css`. This is where text is forced to #000000:

```
/* PDF overrides. Text is forced solid black (#000000, full opacity). */
@page {
  size: Letter;
  margin: 0.85in 0.8in 0.9in;
  @bottom-right {
    content: counter(page) " / " counter(pages);
    font: 500 8.5pt Inter, sans-serif;
    color: #000000;
  }
  @bottom-left {
    content: string(doctitle);
    font: 500 8.5pt Inter, sans-serif;
    color: #000000;
  }
}
@page :first { @bottom-left { content: none; } }
:root {
  --ink: #000000; --muted: #000000; --accent: #0e7490;
  --rule: #94a3b8; --code-bg: #f1f5f9; --bg: #ffffff;
}
html, body { background: #ffffff; }
body {
  max-width: none;
  padding: 0;
  font-size: 10.5pt;
  line-height: 1.55;
  color: #000000;
  font-weight: 400;
}
body, p, li, td, th, h1, h2, h3, h4, blockquote, code, pre,
.subtitle, .byline, .toc a, a {
  color: #000000 !important;
  opacity: 1 !important;
}
.hl * { color: #000000 !important; font-style: normal !important; }

.doc-title { string-set: doctitle content(); font-weight: 700; font-size: 26pt; }
h1 { font-weight: 700; font-size: 20pt; }
h2 { font-weight: 700; font-size: 15pt; break-after: avoid; }
h3 { font-weight: 650; font-size: 12pt; break-after: avoid; }
strong, th { font-weight: 700; }
.subtitle, .byline { font-weight: 500; }
a { text-decoration: none; }

.toc { break-after: page; font-size: 9pt; line-height: 1.35; padding: .6rem 1rem; }
.toc li { font-weight: 500; margin: .05rem 0; }
.toc li.l1 { margin-top: .35rem; }
main h1 { break-before: page; margin-top: 0; }
```

```
blockquote, tr { break-inside: avoid; }
thead { display: table-header-group; }
pre { break-inside: auto; white-space: pre-wrap; } /* wrap at spaces, never mid-token */
pre code { font-size: 8.3pt; }
table { display: table; font-size: 9pt; }
```

Checkpoint: in `uv run python`, run `from rupa.document import parse; from rupa.render import to_pdf; data, n = to_pdf(parse("## Hi")); print(n)`. It prints a page count.

Step 4: The CLI contract

The CLI is where the contract from Part 2 becomes code. Notice that the success path prints exactly one line, and that every failure path has its own exit code.

This listing already includes the `verify` and `refs` subcommands from Step 8. They import `verify.py` only when called, so the CLI works before that file exists.

`src/rupa/cli.py`:

```
"""rupa CLI.

  rupa [INPUT] -o OUT            render Markdown to HTML or PDF
  rupa verify PDF [--wants F]    check a rendered PDF, write review sheets
  rupa refs [--theme T]          rebuild a theme's reference pages

Contract:
  stdin  -> Markdown with optional YAML frontmatter
  file   -> the rendered output (format from -o extension or --to)
  stdout -> ONE status line (or one JSON object with --json). Never the document.
  exit   -> 0 ok, 2 usage, 3 bad input, 4 render failure, 5 verify checks failed
"""

from __future__ import annotations

import argparse
import json
import sys
import tempfile
from importlib import resources
from pathlib import Path

from . import __version__
from .document import InputError, parse
```

```
from .render import RenderError, to_html, to_pdf

EXIT_OK, EXIT_USAGE, EXIT_INPUT, EXIT_RENDER, EXIT_CHECKS = 0, 2, 3, 4, 5
FORMATS = ("html", "pdf")


def _render_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
        prog="rupa", description="Render Markdown to HTML or PDF."
    )
    p.add_argument(
        "input", nargs="?", default="-", help="Markdown file, or - for stdin (default)"
    )
    p.add_argument("-o", "--output", required=True, help="output path, e.g. out.pdf")
    p.add_argument(
        "--to", choices=FORMATS, help="force format instead of using the extension"
    )
    p.add_argument(
        "--theme", default=None, help="theme name (default: frontmatter or 'default')"
    )
    p.add_argument("--json", action="store_true", help="print a JSON status object")
    p.add_argument("--version", action="version", version=f"rupa {__version__}")
    return p


def _err(args, msg: str) -> None:
    if getattr(args, "json", False):
        print(json.dumps({"ok": False, "error": msg}))
    else:
        print(f"rupa: error: {msg}", file=sys.stderr)


def render_main(argv: list[str]) -> int:
    args = _render_parser().parse_args(argv)
    out = Path(args.output)
    fmt = args.to or out.suffix.lstrip(".").lower()
    if fmt not in FORMATS:
        _err(args, f"cannot infer format from '{out.name}'; use --to")
        return EXIT_USAGE

    try:
        text = (
            sys.stdin.read()
            if args.input == "-"
            else Path(args.input).read_text(encoding="utf-8")
        )
        doc = parse(text)
    except (InputError, OSError, UnicodeDecodeError) as exc:
        _err(args, str(exc))
```

```
        return EXIT_INPUT

    theme = args.theme or doc.meta.get("theme", "default")
    try:
        pages = None
        if fmt == "html":
            data = to_html(doc, theme).encode("utf-8")
        else:
            data, pages = to_pdf(doc, theme)
        out.parent.mkdir(parents=True, exist_ok=True)
        out.write_bytes(data)
    except (RenderError, OSError) as exc:
        _err(args, str(exc))
        return EXIT_RENDER

    if args.json:
        print(
            json.dumps(
                {
                    "ok": True,
                    "path": str(out),
                    "format": fmt,
                    "pages": pages,
                    "bytes": len(data),
                }
            )
        )
    else:
        extra = f", {pages} page{'s' if pages != 1 else ''}" if pages else ""
        print(f"rupa: wrote {out} ({fmt}{extra}, {len(data):,} bytes)")
    return EXIT_OK


def verify_main(argv: list[str]) -> int:
    from .verify import EXIT_CHECKS_FAILED, WantsError, load_wants, verify

    p = argparse.ArgumentParser(prog="rupa verify", description="Check a rendered PDF.")
    p.add_argument("pdf")
    p.add_argument("--wants", help="wants.yaml (default: built-in rules)")
    p.add_argument("--theme", help="theme whose reference pages to compare against")
    p.add_argument(
        "--out", help="directory for sheets and verify.json (default: <pdf>.verify/)"
    )
    p.add_argument("--json", action="store_true")
    args = p.parse_args(argv)
    pdf = Path(args.pdf)
    if not pdf.is_file():
        _err(args, f"no such file: {pdf}")
        return EXIT_USAGE
```

```
    try:
        wants = load_wants(args.wants)
    except WantsError as exc:
        _err(args, str(exc))
        return EXIT_INPUT
    out = Path(args.out) if args.out else pdf.with_suffix(".verify")
    report = verify(pdf, wants, out, theme=args.theme)
    if args.json:
        print(Path(report.manifest).read_text())
    else:
        print(report.summary())
    return EXIT_OK if report.ok else EXIT_CHECKS_FAILED


def refs_main(argv: list[str]) -> int:
    from .verify import build_references

    p = argparse.ArgumentParser(
        prog="rupa refs", description="Rebuild a theme's reference pages."
    )
    p.add_argument("--theme", default="default")
    p.add_argument(
        "--out", help="output directory (default: the package's references/<theme>/)"
    )
    args = p.parse_args(argv)
    sample = (resources.files("rupa") / "references" / "sample.md").read_text(
        encoding="utf-8"
    )
    try:
        data, _ = to_pdf(parse(sample), args.theme)
    except RenderError as exc:
        _err(args, str(exc))
        return EXIT_RENDER
    out = (
        Path(args.out)
        if args.out
        else Path(str(resources.files("rupa") / "references" / args.theme))
    )
    with tempfile.TemporaryDirectory() as tmp:
        pdf = Path(tmp) / "sample.pdf"
        pdf.write_bytes(data)
        try:
            picked = build_references(pdf, out)
        except ValueError as exc:
            _err(args, str(exc))
            return EXIT_RENDER
    kinds = ", ".join(f"{k}=p{n}" for k, n in sorted(picked.items()))
    print(f"rupa: wrote references for '{args.theme}' to {out} ({kinds})")
    return EXIT_OK
```

```
def main(argv: list[str] | None = None) -> int:
    argv = list(sys.argv[1:] if argv is None else argv)
    if argv and argv[0] == "verify":
        return verify_main(argv[1:])
    if argv and argv[0] == "refs":
        return refs_main(argv[1:])
    return render_main(argv)


if __name__ == "__main__":  # pragma: no cover
    sys.exit(main())
```

The CLI tests check the contract, not the implementation: one stdout line, real PDF bytes, the black-text rule, each exit code, and blocked remote resources. `tests/test_cli.py`:

```
import io
import json

from rupa.cli import EXIT_INPUT, EXIT_OK, EXIT_RENDER, EXIT_USAGE, main

DOC = "---\ntitle: T\n---\n\n## A\n\nBody text.\n"


def run(monkeypatch, capsys, argv, stdin=DOC):
    monkeypatch.setattr("sys.stdin", io.StringIO(stdin))
    code = main(argv)
    out, err = capsys.readouterr()
    return code, out, err


def test_html_from_stdin(tmp_path, monkeypatch, capsys):
    target = tmp_path / "out.html"
    code, out, _ = run(monkeypatch, capsys, ["-o", str(target)])
    assert code == EXIT_OK
    assert target.read_text().startswith("<!doctype html>")
    # stdout is one status line, never the document
    assert out.count("\n") == 1 and out.startswith("rupa: wrote")


def test_pdf_reports_pages(tmp_path, monkeypatch, capsys):
    target = tmp_path / "out.pdf"
    code, out, _ = run(monkeypatch, capsys, ["-o", str(target), "--json"])
    status = json.loads(out)
    assert code == EXIT_OK
    assert status["ok"] and status["format"] == "pdf" and status["pages"] >= 1
```

```
    assert target.read_bytes()[:5] == b"%PDF-"


def test_pdf_text_is_forced_black(tmp_path):
    from rupa.document import parse
    from rupa.render import to_html

    html = to_html(parse(DOC), target="pdf")
    assert "color: #000000 !important" in html


def test_unknown_extension_is_usage_error(tmp_path, monkeypatch, capsys):
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "out.docx")])
    assert code == EXIT_USAGE and "use --to" in err


def test_bad_input_exit_code(tmp_path, monkeypatch, capsys):
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html")], stdin="")
    assert code == EXIT_INPUT and "empty" in err


def test_unknown_theme_exit_code(tmp_path, monkeypatch, capsys):
    code, _, err = run(
        monkeypatch, capsys, ["-o", str(tmp_path / "o.html"), "--theme", "nope"]
    )
    assert code == EXIT_RENDER and "unknown theme" in err


def test_remote_resources_are_blocked_in_pdf(tmp_path, monkeypatch, capsys):
    md = "## x\n\n![remote](https://example.com/a.png)\n"
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.pdf")], stdin=md)
    assert code == EXIT_RENDER and "example.com" in err


def test_to_flag_overrides_extension(tmp_path, monkeypatch, capsys):
    target = tmp_path / "report.out"
    code, _, _ = run(monkeypatch, capsys, ["-o", str(target), "--to", "pdf"])
    assert code == EXIT_OK and target.read_bytes()[:5] == b"%PDF-"
```

Checkpoint: `uv run pytest -q` shows 16 passed.

Step 5: Install it as a command

```
uv tool install .
echo "# Hello" | rupa -o hello.pdf
```

Expected output:

```
rupa: wrote hello.pdf (pdf, 1 page, …)
```

Try the failure paths too, since Claude will hit them:

```
echo "" | rupa -o x.html          # exit 3: empty input
echo "# x" | rupa -o x.docx       # exit 2: cannot infer format
echo "# x" | rupa -o x.html --theme nope   # exit 4: unknown theme
```

After changing the code, reinstall with `uv tool install --reinstall .`.

Step 6: Continuous integration

CI installs the PDF system libraries, runs the tests, and does a smoke render. `.github/workflows/ci.yml`:

```
name: ci
on: [push, pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: System libraries for WeasyPrint
        run: |
          sudo apt-get update
          sudo apt-get install -y libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz-subset0
      - uses: astral-sh/setup-uv@v6
      - run: uv sync
      - run: uvx ruff check src tests
      - run: uv run pytest -q
      - name: Smoke render
```

```
        run: |
          printf -- '---\ntitle: CI\n---\n\n## ok\n' | uv run rupa -o out/ci.pdf
          uv run rupa verify out/ci.pdf
```

Checkpoint: push to GitHub and confirm the workflow is green.

Step 7: The Claude Code plugin

Create the three plugin files.

`plugin/.claude-plugin/plugin.json`:

```
{
  "name": "rupa",
  "displayName": "Rupa",
  "version": "0.1.0",
  "description": "Write Markdown, let Rupa render the HTML or PDF. Keeps markup out of
Claude's output tokens.",
  "author": { "name": "Vajra Sarpa", "url": "https://vajrasarpa.com" },
  "license": "MIT",
  "keywords": ["render", "pdf", "html", "markdown"]
}
```

`plugin/skills/render/SKILL.md`. The `description` line is the only part loaded every session, so it carries the whole instruction in one sentence:

````
---
name: render
description: Use when the user wants an HTML page or PDF document. Write Markdown, render it
with the rupa CLI, then verify the PDF. Never hand-write HTML, CSS, or PDF code.
---

# Rendering with Rupa

## Render

1. Write the content as Markdown. Optional frontmatter keys: `title`,
   `subtitle`, `author`, `date`, `theme`, `toc` (true/false). No other keys.
2. Use `#` for parts, `##` for sections, `###` for subsections.
   Fenced code blocks take a language tag.
3. Render with one command, using the extension the user asked for:

   ```bash
````
````

````
   rupa draft.md -o out/report.pdf
   ```

4. Read only the one-line status. Do not open or print the output file.
5. On a non-zero exit, read the stderr line and fix the Markdown:
   `3` bad input, `4` render failure.

## Verify (PDF)

1. Run `rupa verify out/report.pdf` (add `--wants wants.yaml` if the project
   has one). It prints one line and writes `verify.json` plus sheet images.
2. Read `verify.json`. Then open each sheet image listed under `sheets`.
   Each sheet shows the theme's reference page on the left and the rendered
   page on the right.
3. Judge every sheet against each item in `rubric`. Record one line per
   sheet: `p<N> <kind>: PASS` or `p<N> <kind>: FAIL <rubric item> - <what>`.
4. Fix failures in the Markdown only: split a wide table, shorten a long
   code line, move or trim content around an awkward break. Then render and
   verify again. Stop after two fix rounds.
5. If a failure can only be fixed in the theme (CSS), do not edit CSS.
   Report it to the user as a theme issue.
6. Tell the user the final check line and your per-sheet verdicts in a few
   lines.

Do not add styling to the Markdown. A different look is a theme change.
````

`plugin/hooks/hooks.json`. It prints a warning at session start only if the CLI is missing:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "command -v rupa >/dev/null || echo 'rupa CLI not on PATH: run uv tool
install <path-to-rupa>'"
          }
        ]
      }
    ]
  }
}
```

Validate and load it:
```

```
claude plugin validate ./plugin
# √ Validation passed

claude --plugin-dir ./plugin
```

Plugin names can't start with `claude-` or `anthropic-`; `rupa` is fine.

Checkpoint: inside that session, ask Claude whether a skill named `rupa:render` is available. It should say yes. Then ask for a one-page PDF on any topic and check that it wrote Markdown and called `rupa`, rather than writing HTML.

Step 8: Verification

`src/rupa/verify.py` holds both tiers. It builds a small page model from the PDF (lines with size, weight and position; fill; table area; share of monospace text), runs the checks on that model, and writes the sheets. Keeping the checks on a plain model rather than on pdfplumber objects is what makes them unit-testable.

Two traps found while building it:

- WeasyPrint paints the page background as one large white rectangle. Left in, it makes every page look 100% full and turns every page into a "table". The code filters it out before measuring.
- The cover page ends in a forced page break after the contents, so it is often short by design. It is exempt from the fill checks, as is the last page.
- A reference page that spills onto a second page is a broken reference: the review compares against half a page. The visual review of this document caught exactly that after a sample edit, so `rupa refs` now refuses a sample whose page for any type runs over.

```
"""Verify a rendered PDF against a wants file.

Two tiers:
  1. Deterministic checks, run here in code: color, size, page count, empty
     pages, stranded headings, overflow. Free and repeatable.
  2. Evidence for a vision review: one side-by-side sheet per page type
     (reference page | rendered page) plus every flagged page. Rupa never calls
     a model; Claude reads the sheets and judges them against the rubric.
"""

from __future__ import annotations
```

```
import json
from dataclasses import asdict, dataclass, field
from importlib import resources
from pathlib import Path

import yaml

PT_PER_IN = 72
PAGE_SIZES = {"letter": (612, 792), "a4": (595, 842)}
# Body box from print.css (@page margins). Footer sits below BODY_BOTTOM.
MARGIN_TOP, MARGIN_SIDE, MARGIN_BOTTOM = 0.85, 0.8, 0.9
FOOTER_CUTOFF = 0.8  # inches from the bottom edge; lines below are footer
PART_HEADING_PT = 18  # an H1 "part" title is at least this size
HEADING_PT = 11.5  # bold, non-mono text at least this size is a heading

ALL_CHECKS = (
    "text-color",
    "page-size",
    "max-pages",
    "blank-page",
    "near-empty",
    "sparse",
    "heading-at-bottom",
    "overflow",
)
DEFAULT_RUBRIC = [
    "hierarchy: titles, headings and body read in the same order and weight as the reference",
    "spacing: margins, gaps and alignment match the reference; nothing crowded or floating",
    "overflow: no text, table or code is clipped, cut off or running into the margin",
    "page balance: no awkward breaks (a lone row, a stranded heading, a near-empty page)",
    "style: colors, rules and boxes match the reference; text is solid black",
]
DEFAULT_WANTS = {
    "theme": "default",
    "page_size": "letter",
    "max_pages": 100,
    "text_color": "#000000",
    "sparse_fill": 0.4,
    "near_empty_fill": 0.2,
    "checks": list(ALL_CHECKS),
    "vision": {
        "sample": "per-type",
        "max_sheets": 8,
        "resolution": 90,
        "rubric": DEFAULT_RUBRIC,
    },
}
EXIT_CHECKS_FAILED = 5
```

```
class WantsError(ValueError):
    """Raised for an unreadable or invalid wants file."""


# ---------------------------------------------------------------- wants


def load_wants(path: str | None) -> dict:
    wants = json.loads(json.dumps(DEFAULT_WANTS))  # deep copy
    if not path:
        return wants
    try:
        given = yaml.safe_load(Path(path).read_text(encoding="utf-8")) or {}
    except (OSError, yaml.YAMLError) as exc:
        raise WantsError(f"cannot read wants file: {exc}") from exc
    if not isinstance(given, dict):
        raise WantsError("wants file must be a YAML mapping")
    unknown = set(given) - set(DEFAULT_WANTS)
    if unknown:
        raise WantsError(f"unknown wants keys: {sorted(unknown)}")
    vision = given.pop("vision", {}) or {}
    unknown_v = set(vision) - set(DEFAULT_WANTS["vision"])
    if unknown_v:
        raise WantsError(f"unknown vision keys: {sorted(unknown_v)}")
    wants.update(given)
    wants["vision"].update(vision)
    bad = set(wants["checks"]) - set(ALL_CHECKS)
    if bad:
        raise WantsError(f"unknown checks: {sorted(bad)}")
    if wants["page_size"] not in PAGE_SIZES:
        raise WantsError(f"page_size must be one of {sorted(PAGE_SIZES)}")
    if wants["vision"]["sample"] not in ("per-type", "all", "none"):
        raise WantsError("vision.sample must be per-type, all or none")
    return wants


# ---------------------------------------------------------------- page model


@dataclass
class Line:
    top: float
    bottom: float
    x0: float
    x1: float
    size: float
    mono: bool
    bold: bool
```

```
    text: str


@dataclass
class PageInfo:
    number: int
    width: float
    height: float
    lines: list[Line] = field(default_factory=list)  # body lines only, no footer
    colors: list[tuple] = field(
        default_factory=list
    )  # one entry per distinct color seen
    off_colors: int = 0  # number of chars not matching the wanted color
    content_bottom: float = 0.0
    table_frac: float = 0.0
    kind: str = "prose"

    @property
    def body_top(self) -> float:
        return MARGIN_TOP * PT_PER_IN

    @property
    def body_bottom(self) -> float:
        return self.height - MARGIN_BOTTOM * PT_PER_IN

    @property
    def fill(self) -> float:
        if not self.lines:
            return 0.0
        span = self.body_bottom - self.body_top
        return max(0.0, min(1.0, (self.content_bottom - self.body_top) / span))

    @property
    def mono_frac(self) -> float:
        chars = sum(len(ln.text) for ln in self.lines) or 1
        return sum(len(ln.text) for ln in self.lines if ln.mono) / chars

    @property
    def starts_part(self) -> bool:
        return bool(self.lines) and self.lines[0].size >= PART_HEADING_PT


def _hex_to_rgb(value: str) -> tuple[float, float, float]:
    v = value.lstrip("#")
    if len(v) != 6:
        raise WantsError(f"text_color must be #RRGGBB, got {value!r}")
    return tuple(int(v[i : i + 2], 16) / 255 for i in (0, 2, 4))
```

```
def _to_rgb(color) -> tuple[float, float, float] | None:
    """Normalize a pdfplumber color (gray, RGB or CMYK tuple) to RGB."""
    if color is None:
        return None
    c = tuple(color) if isinstance(color, (list, tuple)) else (color,)
    if len(c) == 1:
        return (c[0], c[0], c[0])
    if len(c) == 3:
        return c
    if len(c) == 4:
        cy, m, y, k = c
        return ((1 - cy) * (1 - k), (1 - m) * (1 - k), (1 - y) * (1 - k))
    return None


def _same_color(a, b, tol: float = 0.01) -> bool:
    return a is not None and all(abs(x - y) <= tol for x, y in zip(a, b))


def read_pages(pdf_path: str | Path, wanted_color: str = "#000000") -> list[PageInfo]:
    import pdfplumber

    want = _hex_to_rgb(wanted_color)
    pages: list[PageInfo] = []
    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages, start=1):
            info = PageInfo(
                number=i, width=float(page.width), height=float(page.height)
            )
            seen: set = set()
            for ch in page.chars:
                rgb = _to_rgb(ch.get("non_stroking_color"))
                seen.add(rgb)
                if not _same_color(rgb, want):
                    info.off_colors += 1
            info.colors = sorted(c for c in seen if c is not None)
            footer_y = info.height - FOOTER_CUTOFF * PT_PER_IN
            for tl in page.extract_text_lines(return_chars=True):
                if tl["top"] >= footer_y or not tl["chars"]:
                    continue
                chars = tl["chars"]
                fonts = [c.get("fontname", "") for c in chars]
                info.lines.append(
                    Line(
                        top=tl["top"],
                        bottom=tl["bottom"],
                        x0=tl["x0"],
                        x1=tl["x1"],
                        size=max(c["size"] for c in chars),
```

```
                        mono=sum("Mono" in f for f in fonts) > len(fonts) / 2,
                        bold=sum(
                            any(w in f for w in ("Bold", "SemiBold", "Black"))
                            for f in fonts
                        )
                        > len(fonts) / 2,
                        text=tl["text"],
                    )
                )
            body_h = info.body_bottom - info.body_top
            # The page background is painted as one big white rect; ignore it and
            # anything else that spans nearly the whole body.
            rects = [
                r
                for r in page.rects
                if r["bottom"] < footer_y and r["height"] < 0.95 * body_h
            ]
            line_bottoms = [ln.bottom for ln in info.lines]
            info.content_bottom = max(
                [r["bottom"] for r in rects] + line_bottoms, default=0.0
            )
            body_area = (info.width - 2 * MARGIN_SIDE * PT_PER_IN) * body_h
            big = 0.95 * body_h
            no_bg = page.filter(
                lambda o, big=big: (
                    not (o.get("object_type") == "rect" and o["height"] >= big)
                )
            )
            tables = [t for t in no_bg.find_tables() if len(t.columns) >= 2]
            table_area = sum(
                (t.bbox[2] - t.bbox[0]) * (t.bbox[3] - t.bbox[1]) for t in tables
            )
            info.table_frac = min(1.0, table_area / body_area) if body_area else 0.0
            info.kind = classify(info)
            pages.append(info)
    return pages


def classify(p: PageInfo) -> str:
    if p.number == 1:
        return "cover"
    if p.mono_frac > 0.5:
        return "code"
    if p.table_frac > 0.3:
        return "table"
    return "prose"


# ---------------------------------------------------------------- checks
```

```
@dataclass
class CheckResult:
    id: str
    ok: bool
    pages: list[int] = field(default_factory=list)
    detail: str = ""


def run_checks(pages: list[PageInfo], wants: dict) -> list[CheckResult]:
    enabled = wants["checks"]
    results: list[CheckResult] = []
    last = len(pages)

    def add(cid: str, bad: list[int], detail: str) -> None:
        if cid in enabled:
            results.append(CheckResult(cid, not bad, bad, detail if bad else ""))

    add(
        "text-color",
        [p.number for p in pages if p.off_colors],
        f"text not {wants['text_color']}",
    )
    w, h = PAGE_SIZES[wants["page_size"]]
    add(
        "page-size",
        [p.number for p in pages if abs(p.width - w) > 1 or abs(p.height - h) > 1],
        f"page is not {wants['page_size']}",
    )
    add(
        "max-pages",
        list(range(wants["max_pages"] + 1, last + 1)),
        f"{last} pages > max {wants['max_pages']}",
    )
    add("blank-page", [p.number for p in pages if not p.lines], "page has no text")

    near_empty = [
        p.number
        for p in pages
        # The cover (page 1) ends in a forced break and the last page ends the
        # document, so neither is judged on how full it is.
        if p.lines and p.number not in (1, last) and p.fill < wants["near_empty_fill"]
    ]
    add("near-empty", near_empty, f"under {wants['near_empty_fill']:.0%} full")

    sparse = []
    for idx, p in enumerate(pages):
        if not p.lines or p.number in (1, last) or p.number in near_empty:
```

```
            continue
        next_starts_part = idx + 1 < last and pages[idx + 1].starts_part
        if p.fill < wants["sparse_fill"] and not next_starts_part:
            sparse.append(p.number)
    add(
        "sparse",
        sparse,
        f"under {wants['sparse_fill']:.0%} full with no part break after",
    )

    stranded = []
    for p in pages:
        if p.lines:
            tail = p.lines[-1]
            if tail.bold and not tail.mono and tail.size >= HEADING_PT:
                stranded.append(p.number)
    add("heading-at-bottom", stranded, "heading is the last line on the page")

    overflow = []
    for p in pages:
        left = MARGIN_SIDE * PT_PER_IN - 2
        right = p.width - MARGIN_SIDE * PT_PER_IN + 2
        if any(ln.x0 < left or ln.x1 > right for ln in p.lines):
            overflow.append(p.number)
    add("overflow", overflow, "text extends past the side margins")
    return results


# ------------------------------------------------------------------ sampling + sheets


def sample_pages(
    pages: list[PageInfo], results: list[CheckResult], wants: dict
) -> list[PageInfo]:
    """Flagged pages first, then one representative (the middle one) per page kind."""
    mode = wants["vision"]["sample"]
    if mode == "none":
        return []
    if mode == "all":
        return pages[: wants["vision"]["max_sheets"]]
    flagged = sorted({n for r in results for n in r.pages if n <= len(pages)})
    chosen: list[int] = list(flagged)
    for kind in ("cover", "prose", "table", "code"):
        of_kind = [p.number for p in pages if p.kind == kind]
        if of_kind and not any(n in of_kind for n in chosen):
            chosen.append(of_kind[len(of_kind) // 2])
    by_number = {p.number: p for p in pages}
    return [by_number[n] for n in chosen[: wants["vision"]["max_sheets"]]]
```

```
def reference_path(theme: str, kind: str):
    ref = resources.files("rupa") / "references" / theme / f"{kind}.png"
    return ref if ref.is_file() else None


def _label(img, text: str):
    from PIL import Image, ImageDraw, ImageFont

    strip = 28
    out = Image.new("RGB", (img.width, img.height + strip), "white")
    out.paste(img, (0, strip))
    draw = ImageDraw.Draw(out)
    try:
        font = ImageFont.load_default(size=16)
    except TypeError:  # Pillow < 10.1
        font = ImageFont.load_default()
    draw.text((8, 5), text, fill="black", font=font)
    return out


def render_page_png(pdf_path: str | Path, number: int, resolution: int):
    import pdfplumber

    with pdfplumber.open(pdf_path) as pdf:
        return (
            pdf.pages[number - 1]
            .to_image(resolution=resolution)
            .original.convert("RGB")
        )


def make_sheet(
    pdf_path, page: PageInfo, flags: list[str], theme: str, resolution: int, out: Path
) -> Path:
    from PIL import Image

    rendered = render_page_png(pdf_path, page.number, resolution)
    right = _label(
        rendered,
        f"PAGE {page.number} ({page.kind})"
        + (f"  FLAGGED: {', '.join(flags)}" if flags else ""),
    )
    ref = reference_path(theme, page.kind)
    if ref is None:
        sheet = right
    else:
        with resources.as_file(ref) as ref_file:
            ref_img = Image.open(ref_file).convert("RGB")
```

```
            ref_img = ref_img.resize(
                (
                    round(ref_img.width * rendered.height / ref_img.height),
                    rendered.height,
                )
            )
        left = _label(ref_img, f"REFERENCE: {page.kind} ({theme})")
        sheet = Image.new(
            "RGB", (left.width + right.width + 16, right.height), (148, 163, 184)
        )
        sheet.paste(left, (0, 0))
        sheet.paste(right, (left.width + 16, 0))
    path = out / f"sheet-p{page.number:03d}-{page.kind}.png"
    sheet.save(path, optimize=True)
    return path


# ---------------------------------------------------------------- entry point


@dataclass
class VerifyReport:
    pdf: str
    pages: int
    checks: list[CheckResult]
    sheets: list[dict]
    rubric: list[str]
    manifest: str = ""

    @property
    def ok(self) -> bool:
        return all(c.ok for c in self.checks)

    def summary(self) -> str:
        passed = sum(c.ok for c in self.checks)
        fails = [
            f"{c.id} p{','.join(map(str, c.pages[:6]))}"
            for c in self.checks
            if not c.ok
        ]
        line = f"verify: {passed}/{len(self.checks)} checks pass"
        if fails:
            line += f" (FAIL {'; '.join(fails)})"
        if self.sheets:
            line += f"; {len(self.sheets)} sheets for visual review in
{Path(self.manifest).parent}/"
        return line
```

```
def verify(
    pdf_path: str | Path, wants: dict, out_dir: str | Path, theme: str | None = None
) -> VerifyReport:
    theme = theme or wants["theme"]
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    for old in out.glob("sheet-p*.png"):
        old.unlink()
    pages = read_pages(pdf_path, wants["text_color"])
    results = run_checks(pages, wants)
    flags_by_page: dict[int, list[str]] = {}
    for r in results:
        for n in r.pages:
            flags_by_page.setdefault(n, []).append(r.id)
    sheets = []
    for p in sample_pages(pages, results, wants):
        flags = flags_by_page.get(p.number, [])
        path = make_sheet(pdf_path, p, flags, theme, wants["vision"]["resolution"], out)
        sheets.append(
            {"path": str(path), "page": p.number, "kind": p.kind, "flags": flags}
        )
    report = VerifyReport(
        pdf=str(pdf_path),
        pages=len(pages),
        checks=results,
        sheets=sheets,
        rubric=wants["vision"]["rubric"],
        manifest=str(out / "verify.json"),
    )
    Path(report.manifest).write_text(
        json.dumps(
            {
                "pdf": report.pdf,
                "pages": report.pages,
                "ok": report.ok,
                "checks": [asdict(c) for c in results],
                "sheets": sheets,
                "rubric": report.rubric,
            },
            indent=2,
        )
    )
    return report


# ---------------------------------------------------------------- references


def build_references(
```

```
    pdf_path: str | Path, out_dir: str | Path, resolution: int = 90
) -> dict[str, int]:
    """Save the most typical page of each kind in a sample render as <kind>.png.

    Raises ValueError if the sample is missing a page kind, or if a picked page
    spills onto the next page, since a reference must show one whole page.
    """
    pages = read_pages(pdf_path)
    best: dict[str, PageInfo] = {}
    score = {
        "cover": lambda p: 1,
        "code": lambda p: p.mono_frac * p.fill,
        "table": lambda p: p.table_frac * p.fill,
        "prose": lambda p: p.fill,
    }
    for p in pages:
        if p.kind not in best or score[p.kind](p) > score[p.kind](best[p.kind]):
            best[p.kind] = p
    missing = {"cover", "prose", "table", "code"} - set(best)
    if missing:
        raise ValueError(f"sample has no {sorted(missing)} page; extend it")
    for kind, p in best.items():
        nxt = p.number  # pages are 1-based, so this indexes the following page
        if kind != "cover" and nxt < len(pages) and pages[nxt].kind == kind:
            raise ValueError(f"sample {kind} page p{p.number} spills over; trim it")
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    for kind, p in best.items():
        render_page_png(pdf_path, p.number, resolution).save(
            out / f"{kind}.png", optimize=True
        )
    return {kind: p.number for kind, p in best.items()}
```

The sample document every theme's reference pages come from, `src/rupa/references/sample.md`. Each section is sized to fill one page of its type:

```
---
title: Reference Sample
subtitle: The page types every Rupa theme is compared against
author: Rupa
date: Reference
---

## Introduction

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
```

```
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it.

# Part One: Prose

## Body text

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

### A subsection

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

- A list item with **bold** text and `inline code`.
- A second item that runs long enough to wrap onto a second line of the page.
- A third item.

> A blockquote, used for a note or caution in the body of the document.

# Part Two: Tables

## A table page

| Item | Component | Description | Estimate |
|---|---|---|---|
| Item 1 | Component 1 | A short description of what this row covers | 3 hours |
| Item 2 | Component 2 | A short description of what this row covers | 6 hours |
| Item 3 | Component 3 | A short description of what this row covers | 9 hours |
| Item 4 | Component 4 | A short description of what this row covers | 12 hours |
```

Item 5	Component 5	A short description of what this row covers	15 hours
Item 6	Component 6	A short description of what this row covers	18 hours
Item 7	Component 7	A short description of what this row covers	21 hours
Item 8	Component 8	A short description of what this row covers	24 hours
Item 9	Component 9	A short description of what this row covers	27 hours
Item 10	Component 10	A short description of what this row covers	30 hours
Item 11	Component 11	A short description of what this row covers	33 hours
Item 12	Component 12	A short description of what this row covers	36 hours
Item 13	Component 13	A short description of what this row covers	39 hours
Item 14	Component 14	A short description of what this row covers	42 hours
Item 15	Component 15	A short description of what this row covers	45 hours
Item 16	Component 16	A short description of what this row covers	48 hours
Item 17	Component 17	A short description of what this row covers	51 hours
Item 18	Component 18	A short description of what this row covers	54 hours

Part Three: Code

A code page

```python
def step_1(doc):
    """Stage 1 of the sample pipeline."""
    return transform(doc, stage=1)


def step_2(doc):
    """Stage 2 of the sample pipeline."""
    return transform(doc, stage=2)


def step_3(doc):
    """Stage 3 of the sample pipeline."""
    return transform(doc, stage=3)


def step_4(doc):
    """Stage 4 of the sample pipeline."""
    return transform(doc, stage=4)


def step_5(doc):
    """Stage 5 of the sample pipeline."""
    return transform(doc, stage=5)


def step_6(doc):
    """Stage 6 of the sample pipeline."""
    return transform(doc, stage=6)
```

```
def step_7(doc):
    """Stage 7 of the sample pipeline."""
    return transform(doc, stage=7)


def step_8(doc):
    """Stage 8 of the sample pipeline."""
    return transform(doc, stage=8)
```
```

Build the reference pages, and rebuild them whenever the theme changes:

```
uv run rupa refs
# rupa: wrote references for 'default' to .../references/default (code=p5, cover=p1, prose=p3,
table=p4)
```

The wants file with every default spelled out, `wants.example.yaml`. A test keeps it in sync with the code:

```
# What a finished PDF must satisfy. Every key is optional; omitted keys use
# the defaults shown here. Unknown keys are an error.
theme: default            # whose reference pages the sheets compare against
page_size: letter         # letter | a4
max_pages: 100
text_color: "#000000"     # every character must be this color
sparse_fill: 0.4          # flag a page under 40% full, unless a part starts next
near_empty_fill: 0.2      # flag a page under 20% full, always
checks:                   # remove any you don't want
  - text-color
  - page-size
  - max-pages
  - blank-page
  - near-empty
  - sparse
  - heading-at-bottom
  - overflow
vision:
  sample: per-type        # per-type | all | none
  max_sheets: 8           # cap on sheet images per run
  resolution: 90          # DPI for page images; ~1,500 x 1,000 px per sheet
  rubric:                 # what Claude judges on each sheet
    - "hierarchy: titles, headings and body read in the same order and weight as the
reference"
    - "spacing: margins, gaps and alignment match the reference; nothing crowded or floating"
```
```

```
    - "overflow: no text, table or code is clipped, cut off or running into the margin"
    - "page balance: no awkward breaks (a lone row, a stranded heading, a near-empty page)"
    - "style: colors, rules and boxes match the reference; text is solid black"
```

The tests, `tests/test_verify.py`. Most build synthetic pages, so each check is tested in isolation, and two run against a real render:

```
import json

import pytest

from rupa.cli import EXIT_CHECKS, EXIT_INPUT, EXIT_OK, main
from rupa.document import parse
from rupa.render import to_pdf
from rupa.verify import (
    DEFAULT_WANTS,
    Line,
    PageInfo,
    WantsError,
    _to_rgb,
    load_wants,
    run_checks,
    sample_pages,
    verify,
)

LETTER = (612.0, 792.0)


def body_line(top, size=10.5, bold=False, mono=False, x0=60.0, x1=500.0):
    return Line(top, top + size, x0, x1, size, mono, bold, "text")


def page(n, fill_to=700.0, lines=None, **kw):
    lines = lines if lines is not None else [body_line(70), body_line(fill_to - 10.5)]
    p = PageInfo(number=n, width=LETTER[0], height=LETTER[1], lines=lines, **kw)
    p.content_bottom = max((ln.bottom for ln in lines), default=0.0)
    return p


def failing(results):
    return {r.id: r.pages for r in results if not r.ok}


# ---- wants file
```

```
def test_defaults_without_a_file():
    assert load_wants(None) == DEFAULT_WANTS


def test_wants_file_overrides_and_merges_vision(tmp_path):
    f = tmp_path / "wants.yaml"
    f.write_text("max_pages: 3\nvision:\n  max_sheets: 2\n")
    wants = load_wants(str(f))
    assert wants["max_pages"] == 3
    assert wants["vision"]["max_sheets"] == 2
    assert wants["vision"]["sample"] == "per-type"  # untouched default survives


@pytest.mark.parametrize(
    "body, message",
    [
        ("colour: red\n", "unknown wants keys"),
        ("checks: [nope]\n", "unknown checks"),
        ("page_size: a5\n", "page_size"),
        ("vision:\n  model: big\n", "unknown vision keys"),
    ],
)
def test_bad_wants_are_rejected(tmp_path, body, message):
    f = tmp_path / "wants.yaml"
    f.write_text(body)
    with pytest.raises(WantsError, match=message):
        load_wants(str(f))


# ---- colors


def test_color_normalization_handles_gray_rgb_and_cmyk():
    assert _to_rgb((0,)) == (0, 0, 0)
    assert _to_rgb((0.0, 0.0, 0.0)) == (0, 0, 0)
    assert _to_rgb((0, 0, 0, 1)) == (0, 0, 0)
    assert _to_rgb(None) is None


# ---- deterministic checks on synthetic pages


def test_clean_document_passes_everything():
    pages = [page(1), page(2), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_off_color_text_fails():
    pages = [page(1), page(2, off_colors=12)]
```

```
    assert failing(run_checks(pages, load_wants(None))) == {"text-color": [2]}


def test_wrong_page_size_fails():
    pages = [page(1)]
    pages[0].width, pages[0].height = 595.0, 842.0  # A4
    assert "page-size" in failing(run_checks(pages, load_wants(None)))


def test_near_empty_page_fails_but_last_page_is_exempt():
    pages = [page(1), page(2, fill_to=120), page(3), page(4, fill_to=120)]
    assert failing(run_checks(pages, load_wants(None))) == {"near-empty": [2]}


def test_sparse_page_is_allowed_before_a_part_break():
    part = [body_line(70, size=20, bold=True), body_line(700)]
    pages = [page(1), page(2, fill_to=300), page(3, lines=part), page(4)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_sparse_page_without_a_part_break_fails():
    pages = [page(1), page(2, fill_to=300), page(3), page(4)]
    assert failing(run_checks(pages, load_wants(None))) == {"sparse": [2]}


def test_stranded_heading_fails():
    lines = [body_line(70), body_line(690, size=15, bold=True)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {"heading-at-bottom": [2]}


def test_bold_code_at_bottom_is_not_a_heading():
    lines = [body_line(70), body_line(690, size=12, bold=True, mono=True)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_text_past_the_margin_fails():
    lines = [body_line(70), body_line(690, x1=600)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {"overflow": [2]}


def test_disabled_checks_are_skipped():
    wants = load_wants(None)
    wants["checks"] = ["text-color"]
    pages = [page(1), page(2, fill_to=120), page(3)]
    results = run_checks(pages, wants)
    assert [r.id for r in results] == ["text-color"]
```

```
# ---- sampling


def test_sampling_takes_flagged_pages_then_one_per_kind():
    pages = [page(1, kind="cover")] + [page(n, kind="prose") for n in range(2, 9)]
    pages.append(page(9, kind="code"))
    wants = load_wants(None)
    pages[3].off_colors = 1  # page 4 flagged
    picked = [p.number for p in sample_pages(pages, run_checks(pages, wants), wants)]
    assert picked[0] == 4  # flagged first
    assert 1 in picked and 9 in picked  # cover and code represented
    assert sum(pages[n - 1].kind == "prose" for n in picked) == 1  # prose covered once


def test_sampling_none_returns_nothing():
    wants = load_wants(None)
    wants["vision"]["sample"] = "none"
    assert sample_pages([page(1)], [], wants) == []


# ---- end to end on a real render

DOC = """---
title: Verify me
---

## Section

Some body text that renders black.

| a | b |
|---|---|
| 1 | 2 |
"""


@pytest.fixture
def real_pdf(tmp_path):
    data, _ = to_pdf(parse(DOC))
    path = tmp_path / "doc.pdf"
    path.write_bytes(data)
    return path


def test_real_render_passes_and_writes_sheets(real_pdf, tmp_path):
    report = verify(real_pdf, load_wants(None), tmp_path / "v")
    assert report.ok, report.summary()
```

```
    assert report.sheets and all(
        (tmp_path / "v" / s["path"].split("/")[-1]).exists() for s in report.sheets
    )
    manifest = json.loads((tmp_path / "v" / "verify.json").read_text())
    assert manifest["ok"] is True and manifest["rubric"]


def test_cli_verify_exit_codes(real_pdf, tmp_path, capsys):
    assert main(["verify", str(real_pdf), "--out", str(tmp_path / "a")]) == EXIT_OK
    assert capsys.readouterr().out.startswith("verify: ")

    strict = tmp_path / "wants.yaml"
    strict.write_text("max_pages: 1\n")  # cover + TOC page break makes 2 pages
    code = main(
        ["verify", str(real_pdf), "--wants", str(strict), "--out", str(tmp_path / "b")]
    )
    assert code == EXIT_CHECKS
    assert "FAIL max-pages" in capsys.readouterr().out

    bad = tmp_path / "bad.yaml"
    bad.write_text("nope: 1\n")
    code = main(["verify", str(real_pdf), "--wants", str(bad)])
    assert code == EXIT_INPUT


def test_example_wants_file_matches_defaults():
    from pathlib import Path

    example = Path(__file__).parent.parent / "wants.example.yaml"
    assert load_wants(str(example)) == DEFAULT_WANTS


def test_reference_sample_gives_one_whole_page_per_kind(tmp_path):
    from importlib import resources

    from rupa.verify import build_references

    sample = (resources.files("rupa") / "references" / "sample.md").read_text()
    data, _ = to_pdf(parse(sample))
    pdf = tmp_path / "sample.pdf"
    pdf.write_bytes(data)
    picked = build_references(pdf, tmp_path / "refs")
    assert set(picked) == {"cover", "prose", "table", "code"}
    assert all((tmp_path / "refs" / f"{k}.png").exists() for k in picked)


def test_references_refuse_a_spilling_sample(tmp_path):
    from rupa.verify import build_references
```

```
    code = "\n".join(f"x_{i} = {i}" for i in range(200))
    md = f"## Prose\n\nText.\n\n| a | b |\n|---|---|\n| 1 | 2 |\n\n```python\n{code}\n```\n"
    data, _ = to_pdf(parse(md))
    pdf = tmp_path / "s.pdf"
    pdf.write_bytes(data)
    with pytest.raises(ValueError):
        build_references(pdf, tmp_path / "refs")
```

Checkpoint: `uv run pytest -q` shows 40 passed, and `rupa verify hello.pdf` prints a line starting with `verify:` and writes a `hello.verify/` folder with sheets and `verify.json`.

Checkpoint in Claude Code: with the plugin loaded, ask for a one-page PDF and to verify it. Claude should render, run `rupa verify`, open the sheet, give a verdict per sheet, and fix the Markdown if something fails.

Step 9: Measure

`bench/bench.sh` sets up the three arms and runs them. The `awk` line builds the render-only plugin by dropping everything from `## Verify` up to the skill's closing line:

```
#!/usr/bin/env bash
# Compare Claude Code cost on the same task across three arms:
#   baseline       no plugin: Claude writes the HTML/CSS and finds its own PDF tool
#   rupa           plugin, render only (the skill's Verify section is removed)
#   rupa-verify    plugin as shipped: render, then verify with a visual review
#
# Usage: bench/bench.sh [runs]        default 1 run per arm (a pilot)
#        MODEL=claude-sonnet-5-5 bench/bench.sh 3
# Each run starts in a fresh empty directory, so no arm can see this repo.
set -euo pipefail

RUNS="${1:-1}"
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
RESULTS="$ROOT/bench/results/$(date +%Y%m%d-%H%M%S)"
TASK='Write a 2-page PDF report on the AWS Well-Architected security pillar, saved as
report.pdf in the current directory.'
TOOLS="Bash Write Read Edit Skill"
MODEL_FLAG=()
[[ -n "${MODEL:-}" ]] && MODEL_FLAG=(--model "$MODEL")

mkdir -p "$RESULTS"
command -v rupa >/dev/null || { echo "bench: rupa is not on PATH" >&2; exit 2; }

# The render-only arm uses a copy of the plugin whose skill stops before "## Verify".
```

```
NOVERIFY="$RESULTS/plugin-noverify"
cp -r "$ROOT/plugin" "$NOVERIFY"
awk '/^## Verify/ {skip=1} /^Do not add styling/ {skip=0} !skip' \
  "$ROOT/plugin/skills/render/SKILL.md" > "$NOVERIFY/skills/render/SKILL.md"

run_arm() {  # $1 = arm name, remaining args = extra claude flags
  local arm="$1"; shift
  for i in $(seq 1 "$RUNS"); do
    local work; work="$(mktemp -d)"
    echo "bench: $arm run $i" >&2
    ( cd "$work" && claude -p "$TASK" --output-format json --allowedTools "$TOOLS" \
        "${MODEL_FLAG[@]}" "$@" ) > "$RESULTS/$arm-$i.json" 2> "$RESULTS/$arm-$i.err" || true
    if [[ -s "$work/report.pdf" ]]; then
      cp "$work/report.pdf" "$RESULTS/$arm-$i.pdf"
    fi
    rm -rf "$work"
  done
}

run_arm baseline
run_arm rupa --plugin-dir "$NOVERIFY"
run_arm rupa-verify --plugin-dir "$ROOT/plugin"

python3 "$ROOT/bench/summarize.py" "$RESULTS"
```

`bench/summarize.py` reads one results folder and prints the tables:

```
"""Summarize a bench results directory: one row per run, then medians per arm.

Turns and cache reads come first because they drive cost: every turn re-reads
the whole context from cache, while output tokens are a small share.
"""

import json
import statistics
import sys
from pathlib import Path

ARMS = ("baseline", "rupa", "rupa-verify")
COLS = ("turns", "cache_read", "cache_write", "input", "output", "cost_usd", "secs")


def row(path: Path) -> dict | None:
    try:
        r = json.loads(path.read_text())
    except (OSError, json.JSONDecodeError):
        return None
```

```
    u = r.get("usage", {})
    return {
        "turns": r.get("num_turns", 0),
        "cache_read": u.get("cache_read_input_tokens", 0),
        "cache_write": u.get("cache_creation_input_tokens", 0),
        "input": u.get("input_tokens", 0),
        "output": u.get("output_tokens", 0),
        "cost_usd": r.get("total_cost_usd", 0.0),
        "secs": round(r.get("duration_ms", 0) / 1000),
        "pdf": path.with_suffix(".pdf").exists(),
    }


def fmt(v) -> str:
    if isinstance(v, float):
        return f"{v:.3f}"
    if isinstance(v, bool):
        return "yes" if v else "NO"
    return f"{v:,}"


def main(results: Path) -> None:
    runs = {arm: [] for arm in ARMS}
    for f in sorted(results.glob("*.json")):
        arm = f.stem.rsplit("-", 1)[0]
        if arm in runs and (r := row(f)):
            runs[arm].append(r)

    header = ("arm", "run", *COLS, "pdf")
    print("\t".join(header))
    for arm in ARMS:
        for i, r in enumerate(runs[arm], 1):
            print("\t".join([arm, str(i), *(fmt(r[c]) for c in COLS), fmt(r["pdf"])]))

    print("\nmedians (runs without a PDF excluded)")
    print("\t".join(("arm", "n", *COLS)))
    base = None
    for arm in ARMS:
        ok = [r for r in runs[arm] if r["pdf"]]
        if not ok:
            print(f"{arm}\t0\t(no successful runs)")
            continue
        med = {c: statistics.median(r[c] for r in ok) for c in COLS}
        med = {c: v if c == "cost_usd" else round(v) for c, v in med.items()}
        base = base or (med if arm == "baseline" else None)
        print("\t".join([arm, str(len(ok)), *(fmt(med[c]) for c in COLS)]))
    if base:
        print("\nvs baseline (median turns, median cost)")
        for arm in ARMS[1:]:
```

```
            ok = [r for r in runs[arm] if r["pdf"]]
            if ok and base["turns"] and base["cost_usd"]:
                t = statistics.median(r["turns"] for r in ok) / base["turns"] - 1
                c = statistics.median(r["cost_usd"] for r in ok) / base["cost_usd"] - 1
                print(f"{arm}\tturns {t:+.0%}\tcost {c:+.0%}")


if __name__ == "__main__":
    main(Path(sys.argv[1]))
```

Before spending anything, test the plumbing with a stand-in for `claude` that writes a fake `report.pdf` and prints canned JSON. During the build this showed each arm getting the right plugin and the summary computing the comparisons correctly. `bench/fake/claude`:

```
#!/usr/bin/env bash
# Stand-in for `claude` to test bench.sh plumbing without spending tokens.
# Usage: PATH="$PWD/bench/fake:$PATH" bench/bench.sh 2
# Reports fewer turns for the plugin arms so the comparison lines have
# something to show. The numbers are made up; only the plumbing is real.
plugin=""
while [[ $# -gt 0 ]]; do [[ "$1" == "--plugin-dir" ]] && plugin="$2"; shift; done
turns=20
if [[ -n "$plugin" ]]; then
  turns=12
  grep -q "## Verify" "$plugin/skills/render/SKILL.md" && turns=15
fi
printf '%%PDF-1.4 fake\n' > report.pdf
cat <<JSON
{"num_turns": $turns, "total_cost_usd": 0.$turns, "duration_ms": 30000,
 "usage": {"input_tokens": 5, "cache_creation_input_tokens": 50000,
           "cache_read_input_tokens": $((turns * 40000)), "output_tokens": 4000}}
JSON
```

```
PATH="$PWD/bench/fake:$PATH" bench/bench.sh 2
# ...
# rupa         turns -40%   cost -40%
# rupa-verify  turns -25%   cost -25%
```

Those numbers come from the stand-in, not from Claude.

Then run the pilot, one run per arm, and check the transcripts:

```
chmod +x bench/bench.sh
bench/bench.sh
```

If all three arms behaved, run three per arm on a pinned model:

```
MODEL=claude-sonnet-5-5 bench/bench.sh 3
```

Record the medians in your README with the date, model and task. If a Rupa arm isn't cheaper, read its transcript before changing anything else.

Step 10: What to build next

In rough order of value:

1. **Run the pilot, then the benchmark,** and write the result down. Everything else depends on whether Rupa really cuts turns and cost.
2. **A second theme**, such as your dark slate and cyan look for screen HTML, with its own reference pages and a test that renders and verifies every theme.
3. **A golden-render test in CI** that verifies the sample document against its own references, so a CSS change that breaks the look fails the build.
4. **Callouts** (note, warning) through a small markdown-it plugin, so Claude never needs raw HTML for them.
5. **A `--watch` mode** for live preview while you edit by hand.
6. **More targets** (DOCX through python-docx) behind the same contract and exit codes.
7. **Harness integration.** Rupa is a plain CLI, so the Pramā harness can call it the same way Claude Code does.

Turning the build into a showcase

If this becomes a second project about how you built Rupa, capture material as you go rather than reconstructing it afterward.

- **One commit per milestone,** with the milestone number in the message. The git log becomes the outline.
- **A decision log.** The Key decisions table in Part 1 is the start; add a row whenever you change your mind, with the reason.
- **The numbers.** The three-arm tables, with model, date and task. Lead with turns: that's the finding.
- **Before and after artifacts.** The same report produced by hand-written HTML and by Rupa, side by side.
- **The failures.** The WeasyPrint `BaseException` trap, the API change, the background rectangle, and the code wrap that only the visual review caught are exactly the kind of detail that makes a build write-up credible.
- **The verification records.** Each `verify.json` and its sheets show what was checked and what was found. The appendix of this document is one.

A natural shape for a write-up or Loom entry: the problem (each turn re-reads everything, and markup work adds turns), the split (content versus form), the contract, the verification loop, the measurement, and what the numbers showed. The showcase can itself be rendered by Rupa.

Appendix: Verification record

This is the record of verifying this document with the loop it describes. Each pass ran `rupa verify` with the default wants, then a visual review of the sheets against the rubric. The verdicts are Claude's, written one line per sheet as the skill asks.

Pass 1: the first delivered version

The first version ran to 31 pages, before the verification sections existed.

Code checks: `verify: 7/8 checks pass (FAIL near-empty p11,30); 5 sheets`

Sheet	Verdict
p1 cover	PASS
p7 table	PASS
p11 prose	FAIL page balance: one table row (Sustainability) alone on a page before Part 3
p21 code	FAIL overflow: CSS lines broken mid-word, `1px s` / `olid`
p30 prose	FAIL page balance: the last four items of a list alone on a page before the closing part

Two of the three failures were also flagged by the code checks. The third, the mid-word break, was visible only in the image: the text stayed inside the margins, so `overflow` passed.

Fixes. The lone row was a content fix: Cost optimization and Sustainability became one row. The list tail resolved once the new sections moved the page breaks. The mid-word break could only be fixed in the theme, so the loop would have reported it rather than patched it. It was fixed by hand in `print.css` (plain `pre-wrap`), and the long lines in the source files were shortened.

Pass 2: after adding verification

Code checks: `verify: 8/8 checks pass; 4 sheets`

All four rendered pages passed. The review failed the reference instead: the code reference showed the top of a code block that continued onto a sixth page, so the comparison was against half a page. An edit to the sample document had added blank lines. The sample was trimmed, and `rupa refs` now refuses a sample whose page for any type spills over. Two tests cover that.

Final pass

Code checks: `verify: 8/8 checks pass; 4 sheets for visual review in verify/`

Sheet	Verdict
p1 cover	PASS
p10 table	PASS
p12 prose	PASS
p39 code	PASS

Cost of this pass: 4 sheets at about 2,000 input tokens each, against roughly 176,000 tokens to look at all 64 pages at full resolution.